applied sciences

MDPI

Article

Empirical Evaluation of Prompting Strategies for Python Syntax Error Detection with LLMs

Norah Aloufi * and Abdulmajeed Aljuhani

College of Computer Science and Engineering, Taibah University, Medina 41411, Saudi Arabia; abaljuhani@taibahu.edu.sa
* Correspondence: TU4670347@taibahu.edu.sa

Abstract

As large language models (LLMs) are increasingly integrated into software development, there is a growing need to assess how effectively they address subtle programming errors in real-world environments. Accordingly, this study investigates the effectiveness of LLMs in identifying syntax errors within large Python code repositories. Building on the bug in the code stack (BICS) benchmark, this research expands the evaluation to include additional models, such as DeepSeek and Grok, while assessing their ability to detect errors across varying code lengths and depths. Two prompting strategies—two-shot and role-based prompting—were employed to compare the performance of models including DeepSeek-Chat, DeepSeek-Reasoner, DeepSeek-Coder, and Grok-2-Latest with GPT-4o serving as the baseline. The findings indicate that the DeepSeek models generally outperformed GPT-4o in terms of accuracy (Acc). Notably, DeepSeek-Reasoner exhibited the highest overall performance, achieving an Acc of 86.6% and surpassing all other models, particularly when integrated prompting strategies were used. Nevertheless, all models demonstrated decreased Acc with increasing input length and consistently struggled with certain types of errors, such as missing quotations (MQo). This work provides insight into the current strengths and weaknesses of LLMs within real-world debugging environments, thereby informing ongoing efforts to improve automated software tools.

Keywords: large language models (LLMs); syntax error detection; prompt engineering; automated debugging

check for updates

Academic Editor: Stefan Fischer

Received: 26 July 2025
Revised: 18 August 2025
Accepted: 19 August 2025
Published: 21 August 2025

Citation: Aloufi, N.; Aljuhani, A. Empirical Evaluation of Prompting Strategies for Python Syntax Error Detection with LLMs. *Appl. Sci.* **2025**, *15*, 9223. https://doi.org/10.3390/app15169223

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Syntax errors rank among the most common types of programming errors encountered during software development. These errors occur when the written code does not conform to the grammatical rules of the programming language, such as a missing colon (MC), mismatched quotation (MQ), missing parenthesis (MP), or the use of a reserved word as a variable identifier [1]. Although such errors may seem minor, they can seriously disrupt the development process by preventing the code from being properly compiled or interpreted, ultimately hindering the execution of the program. In large-scale software projects that involve thousands of lines of code, syntax errors can impede developers' productivity, as tracking and fixing them often takes a lot of time and can be frustrating [2]. With the advancement of integrated development environments (IDEs) and code analysis tools, new solutions—particularly those powered by artificial intelligence (AI) and machine learning (ML)—are emerging to detect and correct these errors more quickly and efficiently. These

improvements help developers write higher-quality code and enhance the overall efficiency of the development process.

Over the past few years, ML has emerged as a key tool in many areas of software development. For example, in software refactoring, ML algorithms are trained on massive datasets containing millions of actual refactorings from real-world projects. By learning from classes and methods that have undergone refactoring in practice, the resulting models can provide more reliable refactoring recommendations to developers [3]. On the other hand, ML models can improve the detection and correction of syntax errors in software development. Compared to traditional methods, including compiler diagnostics that produce ambiguous or generic messages, ML-based solutions have the potential to provide more useful and automated suggestions. By being trained on massive amounts of syntactically correct code, these models can recognize typical error patterns and support the suggestion of precise fixes [4]. This type of automated assistance has the potential to save developers time and reduce the need for manual debugging.

Building on the foundations of ML, deep learning (DL) has significantly advanced the field of software development by enabling more intelligent and automated solutions to complex programming tasks. For example, in the field of bug detection and correction, neural networks can be trained on massive repositories of code to identify common and subtle programming errors. This approach often outperforms traditional static analysis tools. These DL models are ideally suited for capturing long-term dependencies, such as MP or invalid logical operators, which can be difficult for rule-based systems to detect and are cumbersome for developers to address in cases of very complex code [5]. In addition to bug detection, DL is also used in other areas of software development, such as code refactoring [6], test case generation [7], and code summarization [8]. These capabilities have revolutionized the development workflow, allowing developers to write more secure and efficient code with less manual effort.

As ML technologies advance, LLMs have demonstrated remarkable potential in code-related applications. Originally developed for human language processing, LLMs such as GPT [9] have proven viable in comprehending and generating programming code by virtue of their ability to model both syntax and semantics. Their strength lies in learning from massive, diverse datasets, which allows them to perform tasks such as code completion [10], bug detection [11], and program synthesis [12]. Recent benchmarks such as needle-in-a-haystack (NIAH) [13] and bug in the code stack (BICS) [14] evaluate how effectively these models can locate subtle bugs or retrieve relevant information across large codebases. As LLMs continue to improve, they are expected to play a more significant role in enhancing developer productivity through real-time suggestions and intelligent debugging assistance.

Several research studies have analyzed the process of extracting critical small data points from extensive text collections by using the NIAH benchmark to evaluate LLMs on their ability to locate such information within extensive texts. NIAH specializes in retrieving text, but researchers now utilize LLM technology for software-related projects by evaluating systems based on their capacity to detect errors in codebases. BICS provides a systematic analysis of LLM debugging performance with Python code as an evaluation platform. The BICS benchmark system creates syntax errors at different depths within extensive code stacks to evaluate GPT-4o, Claude 3 [15], Gemini 1.5 [16], and additional models regarding their error discovery competencies. Research results demonstrate how longer context affects system performance, since certain models experience a decline in performance when the code length increases.

The contribution of this paper is to investigate the effectiveness of various LLMs, such as DeepSeek [17] and Grok [18], in detecting syntax errors under varying code lengths, code depths, and prompt designs. It further explores the impact of prompt engineering on enhancing model performance. It also highlights the critical limitations in current LLMs for accurate syntax error detection and their effectiveness in a real-world setting. Finally, the paper provides insights and recommendations for future improvements in model capabilities.

2. Related Work

Several studies have addressed the use of ML techniques to identify code errors—particularly syntax-related issues [19–21]. For example, through automatic code bug fixing with unsupervised learning, Yasunaga and Liang [22] proposed break-it-fix-it (BIFI), a method that trains a fixer to correct bad code (e.g., syntax errors) and a breaker to generate realistic bad code, thereby iteratively improving both models without the need for labeled data. The authors highlight the limitations of existing approaches that rely on synthetic perturbations, which often fail to match real error distributions, and demonstrate BIFI's effectiveness on two datasets: GitHub-Python and DeepFix. With a Transformer-based model, BIFI attained state-of-the-art results with 90.5% accuracy (Acc) on GitHub-Python and 71.7% on DeepFix, surpassing baseline methods such as back-translation. This paper also covers BIFI's real-world error pattern adaptation strategies using a critic (e.g., a compiler) to verify corrections, in the context of unsupervised machine translation (MT) and domain adaptation.

By helping beginners fix Java syntax errors, Ahmed et al. [23] presented SynShine, a tool that combines compiler error messages (javac), pre-trained language models (RoBERTa), and smart editing suggestions to fix mistakes. The authors show how existing tools struggle with long code and unclear errors, while SynShine achieves 75% Acc on single-error fixes, beating prior tools like BF + FF by 18%. The key steps are: (1) use javac to help narrow down the bugs, (2) harness RoBERTa for learning code gist, and (3) generate fixes through simple "insert/delete/replace" operations instead of complicated code rewriting. As a part of VSCode, SynShine is fast even on basic computers.

Combining context-aware neural translation (CoCoNuT), a unique automatic program repair (APR) technique, is presented by Lutellier et al. [24]. It fixes bugs in several programming languages by utilizing context-aware neural machine translation (NMT) and ensemble learning. By combining convolutional neural networks (CNNs) for hierarchical feature extraction with a dual-encoder NMT architecture that evaluates buggy code and its context independently, the authors overcome the drawbacks of conventional APR techniques, such as their dependency on hard-coded rules. Their method accurately fixes 509 bugs (including 309 that were previously unfixed) in Java, C, Python, and JavaScript benchmarks, outperforming 27 state-of-the-art techniques. Although the use of attention maps facilitates interpretability, limitations include difficulties with generalizability outside of tested benchmarks, and the computational expense of training.

Hoppity, a learning-based tool for detecting and fixing JavaScript bugs using graph transformations, is presented by Dinella et al. [25]. By modeling issue patches as a series of graph changes (such as adding, removing, or altering nodes in a program's abstract syntax tree (AST)), the authors present a novel method. Unlike other works, Hoppity works end-to-end, locating bugs and producing fixes without the need for human participation, and it manages complicated bugs that call for structural modifications. The model, which was trained on 290,715 JavaScript code change commits on GitHub, combines an LSTM-based controller to forecast modifications with a graph neural network (GNN) for program representation. According to the results, Hoppity outperforms baselines like gated graph

neural networks (GGNN) and SequenceR, correctly fixing 9490 out of 36,361 programs. Limitations include scalability to large ASTs and reliance on beam search for inference.

In the field of bug fixing, tools struggle to accurately cover a wide range of bugs due to the complexity and size of modern code databases. To address this challenge, Berabi et al. in [26] introduced a new learning-based system, TFix, which presents the bug fixing problem as a text-to-text task, enabling it to operate directly on the program text and avoid complex representations. They did this by leveraging a powerful T5-based Transformer model, pre-trained on natural language and then fine-tuned to generate code fixes using a large, high-quality dataset of 5.5 million commits on GitHub. All 52 bug types were fine-tuned to enhance knowledge transfer. The findings demonstrated that TFix works well in reality, producing code that corrects errors in roughly 67% of instances and surpassing other learning techniques like SequenceR, CoCoNuT, and Hoppity by a significant margin. The limitations of this approach include the fact that the exact match metric represents a lower bound on Acc due to the possibility of multiple ways of fixing the error, and future work is proposed to extend TFix to other languages and more complex error types where error localization is difficult.

Recently, the focus has started to shift from traditional ML approaches to LLMs, which offer broader capabilities in detecting and fixing bugs in code. For example, by automating the generation of feedback for syntax errors within Python scripts, Phung et al. [27] explore how LLMs, in particular OpenAI's Codex, might improve programming education. By developing a method called PyFiXV, the authors aim to support educators in monitoring the level of feedback that learners receive. The shortcomings of conventional programming-context messages are pointed out, and the difficulties faced by inexperienced programmers when they run into syntax errors are highlighted.

Applying the naturalness hypothesis to pre-trained code LLMs, Campos et al. [11] introduced a lightweight method for bug detection and localization in source code based on token-level likelihood scores. The authors hypothesize that LLMs, trained predominantly on correct code, assign lower likelihoods to buggy code, which they formalize through two metrics: likelihood score (mean token probability) and likelihood ratio (comparison to the model's preferred token). Assessing the public HumanEvalFix benchmark, as well as the private Subato benchmark containing student submissions, the authors show that smaller models like Code Llama 7B can achieve Acc levels of 78% on bug detection, with token-level error localization at 79% (top three tokens). However, the study points out important shortcomings, such as a drop in performance due to overfitting, data leaks, unfamiliar coding styles, and other peculiarities.

By automating feedback generation for syntax and runtime errors in Python scripts, Yang et al. [28] introduced a data science debugging benchmark (DSDBench), a benchmark specifically designed to evaluate how well LLMs debug complex data science code with multiple bugs and multi-hop logical errors. The authors built DSDBench using code samples from existing benchmarks like DABench and MatPlotBench, injecting both single- and multi-error scenarios through LLM-based methods, and carefully annotating cause-effect lines and error messages. They tested several state-of-the-art models, including GPT-4o, Claude, and DeepSeek-R1, and found that while models perform reasonably well on single-bug detection, their Acc significantly drops in multi-bug cases, especially when tracing the root cause across multiple lines of code. The benchmark highlights how existing LLMs struggle with real-world data science debugging tasks and emphasizes that there is still more work to be performed to enhance an LLM's reasoning skills for intricate and multiple-file codebases.

Prenner and Robbes [29] take a closer look at whether Codex can fix bugs in software without needing specific training for that task. They run Codex on 40 buggy programs from the QuixBugs benchmark, written in Python and Java, using different prompt styles like code-only, code-with-hint, code-with-docstring, input-output examples, and others. They show that Codex is able to repair 23 Python bugs (outperforming CoCoNuT and DeepDebug) and 14 Java bugs (though with lower precision in Java), even though the model has not been trained explicitly for APR. Major limitations include possible data leakage-Codex may have seen QuixBugs during training-and the reliance on manual evaluation. The paper demonstrates the potential of Codex for APR, but suggests that further exploration is needed through automation, parameter fine-tuning, and wider benchmarking.

Lee et al. [14] evaluated the ability of LLMs to detect simple syntactic errors in large Python code structures, where information retrieval in software environments faces significantly greater challenges than in text-based environments. To address this gap, the researchers introduce a new benchmark called BICS, the first of its kind that specifically targets the debugging capabilities of long-context code. BICS is constructed by compiling clean Python code snippets from the Alpaca dataset, after removing existing errors using a static code analyzer, to form "code stacks" of varying token lengths ranging from 500 to 16,000 tokens. One syntactic error of seven specific types is then injected into one of these stacks at five different target depths (0%, 25%, 50%, 75%, and 100%). Eleven leading LLMs (such as GPT-4o and Claude 3.5 Sonnet) were evaluated by asking them to accurately identify the line number and error type, which requires a deep understanding of the code structure. The results revealed that code environments pose significantly more challenges for retrieval tasks than text-based environments, with significant variation in performance between models. GPT-4o and GPT-4-Turbo performed the best, with an average accuracy (Avg. Acc) of 81.2% for GPT-4o. A significant degradation in performance was also observed with increasing context length, and performance varied depending on error depth and type, with the MC and MP errors being the most easily retrievable. The benchmark's current shortcomings include its emphasis on basic syntactic errors; however, future research suggests broadening its scope to encompass runtime errors, warnings, and a variety of programming languages, including C++ and Rust.

The innovation of this study lies in its systematic exploration of how different LLMs handle syntax error detection in long and complex code structures. While earlier research typically evaluated model performance using a single prompting strategy, this study broadens the scope by examining multiple LLMs across varied prompt designs, code lengths, and error depths. This approach enables a more comprehensive assessment of model capabilities and robustness, better reflecting real-world programming scenarios. By combining model-level comparisons with analyses of prompt strategies, the work not only identifies performance gaps but also uncovers critical limitations in stability and consistency, offering a nuanced understanding of LLM capabilities in software debugging contexts.

3. Framework

This research investigates the ability of LLMs to detect syntax errors in substantial Python codebases with more than 16,000 lines. Extending the BICS benchmark [14], we introduce Python scripts with intentional syntax errors—specifically, MC and mismatched brackets (MB)—at differing depths in the code. The paper analyzes the performance of three DeepSeek models (DeepSeek-Chat, DeepSeek-Reasoner, and DeepSeek-Coder) alongside Grok-2-Latest under two prompting paradigms: two-shot prompting and role-based prompting. We subsequently compare the performance of the models with the benchmark GPT-4o, as established by the BICS. To facilitate an in-depth analysis, we

compare the models based on the following important metrics: Acc, standard deviation (STD), and the duration taken to run the complete evaluation.

The proposed framework is divided into several stages, starting from cleaning the dataset, then preprocessing, such as generating the prompt and preparing the test parameters, to predicting the results using the model, as depicted in Figure 1.

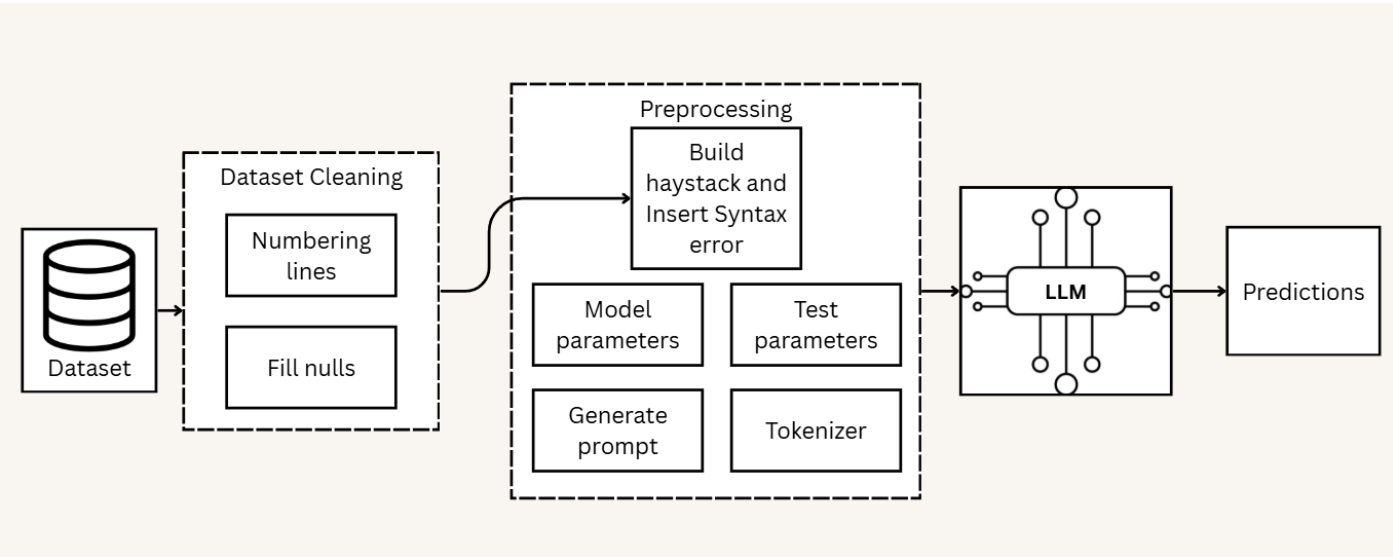

Figure 1. Proposed framework.

3.1. Dataset

The dataset used for this study is the Alpaca dataset [30], a collection of 18,000 Python code samples with accompanying text commands. It is primarily designed for code generation tasks based on text instructions, with each sample containing a textual description of the task and its corresponding code. The dataset encompasses a broad range of programming concepts, including functions, loops, conditionals, list comprehensions, dictionaries, classes, and imports. This diversity makes it a comprehensive resource for learning and identifying syntax errors in Python. Example tasks range from basic operations like sequence summation or palindrome checking to advanced ones like password randomness generation or making web requests, with the dataset being highly valuable for both educational use and automated code analysis. In this paper, a modified version of the dataset was used and preprocessed to match the requirements of the experiment prior to use with the LLM, which was Python code snippets with corresponding altered versions that contain a specific syntax error. The samples are constructed to embody typical programming errors and their precise positions within the code. Each sample contains the correct code, along with variants that have errors, including but not limited to MC, MB, MP, MQo, MCo, and incorrect usage of reserved keywords as identifiers (KID). Line numbers in the corresponding places straightforwardly show the locations of all errors. Null values in the dataset were filled, and a line numbering function was created, which will later be used to identify the line where the syntax error occurred.

3.2. Preprocessing

Preprocessing is one of the most important stages of data preparation for training LLMs. Since LLMs rely heavily on the quality and structure of their input data, effective preprocessing ensures that the model learns valuable patterns and generates coherent output. Without proper preprocessing, the model may struggle with inconsistencies or irrelevant content, leading to reduced performance and poor generalization. One essential preprocessing step applied in this study was tokenization, which splits text into smaller

units called tokens. Tokenization improves the capacity of the model to understand and process language more effectively. The input data are made more structured and compatible with the model architecture [31]. Another important preprocessing step involves building a haystack and inserting the syntax error. This function builds a stack of Python code according to the target length and inserts syntax errors according to the target depth, as defined by the test parameters, in addition to the syntax error types outlined in Section 3.2.1. The model parameters, on the other hand, include the API key, which is a unique authentication token that grants authorized access to LLM services via their API endpoints. The context size of the model is used to ensure that the entire prompt does not exceed the context size.

3.2.1. Syntax Errors

In Python, several common syntax errors can disrupt code execution, each with distinct characteristics and causes. A list of the syntax errors addressed in this experiment is listed in Table 1. These errors can be identified by careful code review, using tools, or leveraging IDE features that highlight syntax issues, such as ensuring proper punctuation, matching delimiters, and avoiding reserved words.

Table 1. Syntax error types.

No.	Syntax Error	Description
1	Missing colons (MC)	Occurs when the required (:) is omitted at the end of control structures such as function definitions, loops, or conditionals—for example, def my_function() without the colon—which prevents Python from recognizing the code block.
2	Missing parenthesis (MP)	Happens when parentheses () are not properly closed in function calls, expressions, or definitions, like print (“Hello” without a closing parenthesis, leading to unbalanced syntax.
3	Missing quotation (MQo)	Arises when string literals lack matching opening and closing quotation marks (“ or ’), such as name = “John without a closing quote, confusing string boundaries.
4	Missing comma (MCo)	Triggered when commas are omitted in sequences or argument lists, like [1 2 3] instead of [1, 2, 3], disrupting element separation.
5	Mismatched quotation (MQ)	Occurs when opening and closing quotes do not match, such as “Hello’ using different types, causing parsing issues.
6	Mismatched brackets (MB)	Results from incorrect or unbalanced brackets, like [1, 2, 3} instead of [1, 2, 3], which confuses data structure definitions.
7	Keyword as identifier (KID)	Naming a variable def = 10, is invalid because keywords like def are reserved, leading to conflicts.

3.2.2. Prompt Engineering

We assess each model under two prompting strategies: two-shot prompting and role-based prompting. Two-shot prompting is a technique in which the model is given two examples of solving a particular type of problem or question before being asked to solve a new example. The goal is to guide the model towards its solution by learning from

the provided examples [32]. We provide two examples of syntax errors before the target task, as shown in Figure 2.

```
I will give you a codebase with a syntactic bug hidden at some line.
You need to answer the line at which the syntactic error occurs and the type of the syntactic error.

1 | def fahrenheit_to_celsius(fahrenheit):
2 |   return (fahrenheit - 32) * 5.0/9.0
3 |
4 | def is_prime(num:
5 |     if num <= 1:
6 |         return False
7 |     for i in range(2, int(num**0.5) + 1):
8 |         if num % i == 0:
9 |             return False
10|     return True
Answer: 4, missing_parenthesis

1| import random
2| import string
3|
4| def generate_password(length=8):
5|     characters = string.ascii_letters + string.digits
6|     password = '\''.join(random.choice(characters) for i in range(length))
7|     return password
Answer: 6, mismatched_quotation

{background_code}

missing_colon
missing_parenthesis
missing_quotation
missing_comma
mismatched_quotation
mismatched_bracket
keywords_as_identifier

Always return your answer in the following format: <line_number>, <bug_type>
Do not write anything else after that.
```

Figure 2. Example of a two-shot prompt used in model evaluation.

Role-based prompting is a technique that leverages the inherent ability of LLMs to adopt various personas, providing them with a specific identity and context to improve their performance [33]. In this study, we assign a role before presenting the two-shot examples, as shown in Figure 3.

```
You are a member of a Python application development project team.
Your task is to analyze a given codebase and identify the line number that contains a syntax error
along with the type of the error. Below are some examples of how errors should be detected:

Example 1:
1 | def fahrenheit_to_celsius(fahrenheit):
2 |   return (fahrenheit - 32) * 5.0/9.0
3 |
4 | def is_prime(num:
5 |     if num <= 1:
6 |         return False
7 |     for i in range(2, int(num**0.5) + 1):
8 |         if num % i == 0:
9 |             return False
10|     return True
Answer: 4, missing_parenthesis

Example 2:
1| import random
2| import string
3|
4| def generate_password(length=8):
5|     characters = string.ascii_letters + string.digits
6|     password = '\''.join(random.choice(characters) for i in range(length))
7|     return password
Answer: 6, mismatched_quotation

{background_code}

missing_colon
missing_parenthesis
missing_quotation
missing_comma
mismatched_quotation
mismatched_bracket
keywords_as_identifier

Always return your answer in the following format: <line_number>, <bug_type>
Do not write anything else after that.
```

Figure 3. Example of a role-based prompt used in model evaluation. ** represents exponentiation in python code.

3.3. LLM Models

Four LLMs were tested and compared against BICS, including three from DeepSeek and one from xAI, as shown in Table 2. LLMs were prompted not only to determine whether there was a syntax error, but also to predict the specific type of syntax error and the exact line number where the error occurred. This approach simulates real debugging scenarios, where programmers rely on their tools to provide both locational and descriptive information.

Table 2. LLMs evaluated in the experiment.

No.	Model Name	Publisher	Context Size	Latest Released
1	DeepSeek-Chat	DeepSeek AI	64k	2024
2	DeepSeek-Reasoner	DeepSeek AI	64k	2025
3	DeepSeek-Coder	DeepSeek AI	128k	2024
4	Grok-2-Latest	xAI	128k	2024

4. Experiment

In this experiment, we tested the models at different lengths and depths. Length refers to the desired size of the combined code snippets—specifically, the total token count of the final concatenated string—which was tested with values [500, 1k, 2k, 4k, 8k, 16k], measured using a tokenizer to ensure the output aligns with LLM input limits and various testing scenarios. Depth, tested at [0%, 25%, 5%, 75%, 100%], indicates the proportion of that target length filled with initial correct code before inserting a specific error, thus controlling the relative position of the error within the overall structure. This is how the haystack was built and the needle (syntax error) inserted. Together, these parameters enable us to investigate a range of conditions by adjusting the total code length and the error's placement, and following each prompt delivery, we conducted 25 test iterations for each combination of code length (e.g., 8k tokens) and error depth (e.g., 50% depth), resulting in 750 total test runs per model and prompt strategy (6 lengths × 5 depths × 25 iterations). This process systematically combined these values and involved repeating the procedure multiple times per combination to gather thorough and reliable insights into their impact. For example, when the length is 1k and the depth is 50%, it was tested 25 different times, each time with a different haystack and type of error. Each time, the model was fed a prompt containing the haystack and the error, ensuring that the entire prompt remained within the model's window. All experiments were conducted using Google Colab and Python 3.12.11 to ensure consistent execution across models and test iterations.

The performance of the model was evaluated based on three main evaluation metrics: Acc, STD, and execution time. Acc measures how many times the model predicted correctly; it is a proportional measure of the number of correct predictions (true positives (TP) and true negatives (TN)) to all predictions (TP, TN, false positive (FP) and false negative (FN)), as presented in Equation (1).

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \tag{1}$$

STD measures how consistently the model predicts correctly. It is a statistical metric that quantifies the degree to which individual predictions deviate from the average prediction. A lower STD indicates more consistent predictions, whereas a higher STD reflects greater variability. It is calculated as the square root of the average of the squared differences between each prediction and the mean, where x_i represents each data point, μ is the mean of the data, and N is the total number of data points, as presented in Equation (2).

$$\text{STD} = \sqrt{\frac{\sum_{i=1}^{N}(x_i - \mu)^2}{N}} \tag{2}$$

Execution time refers to the amount of time it takes for a model to complete a specific task or prediction. It is a crucial performance metric, particularly when comparing models in terms of efficiency and usability in real-time applications. A shorter execution time indicates a faster and more efficient model, which is beneficial for time-sensitive applications. In this evaluation, the time taken by each model during testing was measured and compared to assess their computational efficiency.

5. Results

This section is divided into three subsections. The first presents the results obtained using two-shot prompting, while the second focuses on role-based prompting combined with two-shot prompting. For each setting, the evaluation is based on two Acc metrics: the Avg. Acc per input length, calculated by averaging the model's performance across all error depths, and the overall Avg. Acc, which represents the Avg. Acc of the model across all lengths and error types. The last subsection highlights the computational time taken by each model to complete the experiment. These metrics allow for an overall comparison of model performance in terms of code size, complexity, STD, and the time required for each to complete all iterations.

5.1. Two-Shot Prompting

In this experiment using two-shot prompting, we evaluated four LLMs: DeepSeek-Chat, DeepSeek-Reasoner, DeepSeek-Coder, and Grok-2-Latest, across varying target code lengths and depths, with 25 iterations per combination, yielding a total of 750 test runs per model to assess their Acc in handling code with embedded errors. The results reveal distinct performance trends: the DeepSeek models consistently achieved high Avg. Acc, ranging from 83.3% to 83.6%, but exhibited a decline as token length increased, as shown in Figure 4. DeepSeek-Chat dropped from 85.6% at 500 tokens to 72.0% at 16k, and DeepSeek-Reasoner from 92.8% to 66.4%, suggesting sensitivity to longer contexts despite their large windows. This was accompanied by a high STD from ±37.0 to ±37.3, indicating variability across depths and lengths. DeepSeek-Coder maintained relatively stable performance, ranging from 88.8% to 72.8%, slightly outperforming its counterparts, likely due to its code-specific design. In contrast, Grok-2-Latest, exhibited the lowest Avg. Acc of 50.2%, with a sharp decline from 64.0% at 500 tokens to 36.0% at 16k, and the highest STD ±50.0, reflecting inconsistent performance and potential struggles with error detection in longer or more complex contexts. For benchmarking, GPT-4o demonstrated a balanced Avg. Acc of 81.2% with a low STD of ±5.5, maintaining steady performance from 82% to 74% across all lengths, which highlights its robustness and reliability. Overall, while the DeepSeek models excelled at shorter lengths, Grok-2-Latest's weaker performance and GPT-4o's consistent results underscore the varying capabilities of these models in processing extended code contexts with embedded bugs. Table 3 shows the results for the two-shot prompting setting.

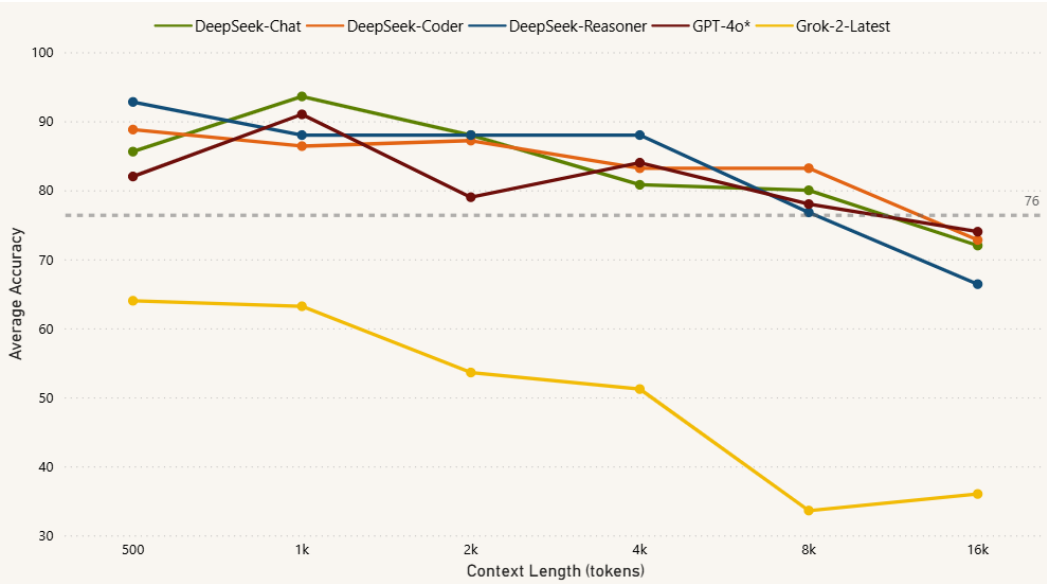

Figure 4. Acc of models using two-shot prompting across different input lengths, *GPT-4o [14].

Additionally, we analyzed model performance across error types using two-shot prompting. DeepSeek-Chat achieved an Avg. Acc of 83.3%, with strong performance on MC at 95.6% and MB at 94.0%, but weaker results on MQ at 67.4%. DeepSeek-Reasoner, also averaging 83.3%, excelled on KID at 89.8% and MCo at 88.7%, yet struggled with MQo at 51.3%. DeepSeek-Coder, with the highest Avg. Acc of 83.6%, performed best on MP at 95.8% and MC at 94.0%, though it dipped to 73.1% on MQ, demonstrating robustness across most error types. Grok-2-Latest, averaging 50.2%, showed inconsistent results, with a peak on MC at 82.1% but notably poor performance on MP at 29.1% and MQ at 30.9%, reflecting challenges with certain syntax errors. For comparison, the GPT-4o benchmark from prior research averaged 81.2%, with high marks on MC at 93%, MP at 93%, and MB at 96%, but a significant drop on MQo at 18%, suggesting variability in error-handling capabilities. These findings highlight that while DeepSeek models generally handle diverse errors well, Grok-2-Latest struggles with specific syntax issues, and GPT-4o provides a reliable baseline—each model's performance varies by error type under two-shot prompting conditions. Table 4 presents the results for each error type under the two-shot prompting setup.

Table 3. Acc scores of models with two-shot prompting at varying input lengths.

No.	Model	500	1k	2k	4k	8k	16k	Avg. Acc	STD
1	DeepSeek-Chat	85.6	93.6	88.0	80.8	80.0	72.0	83.3	±37.3
2	DeepSeek-Reasoner	92.8	88.0	88.0	88.0	76.8	66.4	83.3	±37.2
3	DeepSeek-Coder	88.8	86.4	87.2	83.2	83.2	72.8	83.6	±37.0
4	Grok-2-Latest	64.0	63.2	53.6	51.2	33.6	36.0	50.2	±50.0
5	GPT-4o [14]	82	91	79	84	78	74	81.2	±5.5

Table 4. Acc of each model across error types under two-shot prompting.

No.	Model	MC	MP	MQo	MCo	MQ	MB	KID	Avg. Acc
1	DeepSeek-Chat	95.6	92.1	76.5	80.0	67.4	94.0	73.1	83.3
2	DeepSeek-Reasoner	90.3	87.5	51.3	88.7	82.4	94.1	89.8	83.3
3	DeepSeek-Coder	94.0	95.8	73.9	85.1	73.1	91.0	75.9	83.6
4	Grok-2-Latest	82.1	29.1	54.0	60.0	30.9	38.1	58.8	50.2
5	GPT-4o [14]	93	93	18	90	91	96	80	81.2

5.2. Role-Based and Two-Shot Prompting

In a separate evaluation using role-based and two-shot prompting, we tested DeepSeek-Chat, DeepSeek-Reasoner, DeepSeek-Coder, Grok-2-Latest, and GPT-4o across the same target lengths and depths, again with 750 test runs per model. The results with this combined prompting strategy showed improved performance trends: DeepSeek-Chat achieved an Avg. Acc of 83.8%, with scores ranging from 87.2% at 500 tokens to 68.8% at 16k; STD was ±36.8. DeepSeek-Reasoner reached the highest average of 86.6%, with a peak of 92.0% at 1000 tokens and a low of 78.4% at 16k; STD was ±34.0. DeepSeek-Coder averaged 81.4%, ranging from 88.0% at 1000 tokens to 67.2% at 16k; STD was ±38.9, as shown in Figure 5. Compared to two-shot prompting alone, role-based prompting enhanced overall accuracies, particularly for DeepSeek-Reasoner, and mitigated some decline at longer lengths, e.g., DeepSeek-Chat from 72.0% to 68.8% at 16k, though variability remained high. This suggests that adding role-based context to two-shot prompting bolsters error detection across diverse code lengths, with DeepSeek-Reasoner showing the most consistent gains, while still reflecting challenges with extended contexts, as indicated by the persistent drop-off at 16k tokens across all models. Grok-2-Latest achieved an Avg. Acc of 59.3%, ranging from 80.8% at 500 tokens to 36.8% at 16k, with a STD of ±49.1. GPT-4o reached an Avg. Acc of 73.8%, with scores from 80.8% at 500 tokens to 64.0% at 16k, and STD ±43.9. Table 5 shows the results with role-based and two-shot prompting.

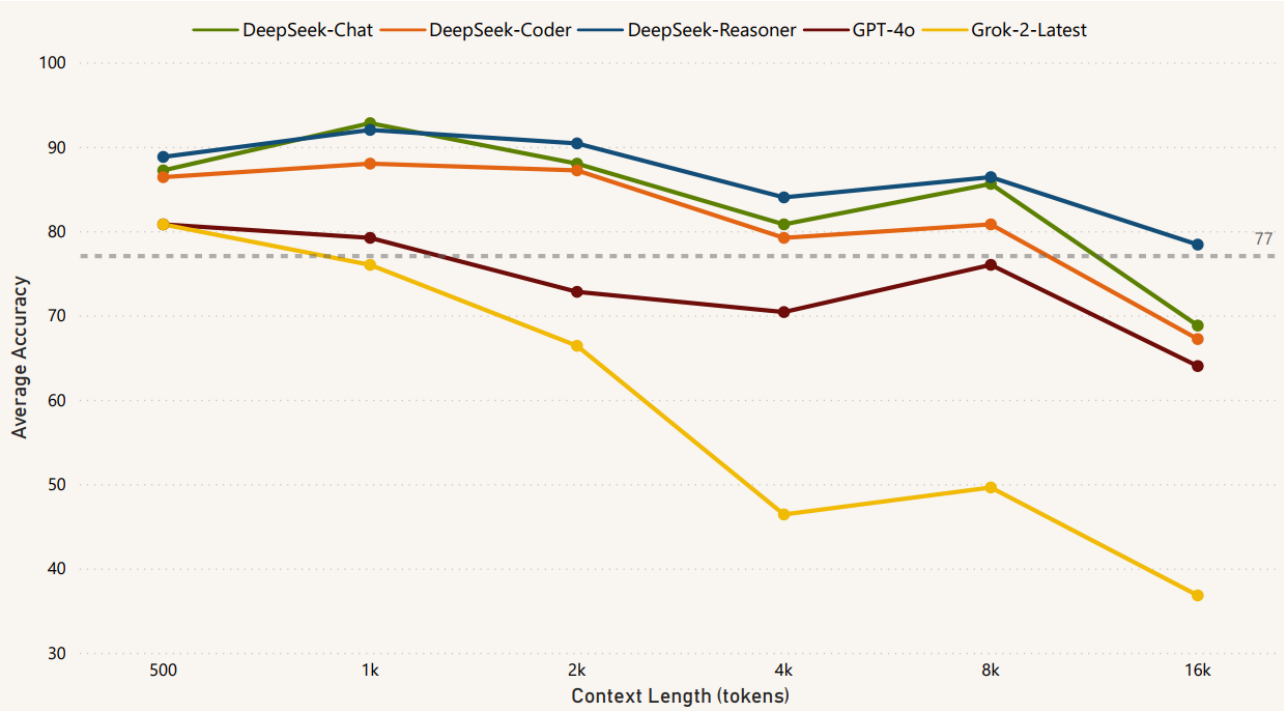

Figure 5. Acc variation across models and input lengths under role-based and two-shot prompting.

Table 5. Acc scores for each model using role-based and two-shot prompting.

No.	Model	500	1k	2k	4k	8k	16k	Avg. Acc	STD
1	DeepSeek-Chat	87.2	92.8	88.0	80.8	85.6	68.8	83.8	±36.8
2	DeepSeek-Reasoner	88.8	92.0	90.4	84.0	86.4	78.4	86.6	±34.0
3	DeepSeek-Coder	86.4	88.0	87.2	79.2	80.8	67.2	81.4	±38.9
4	Grok-2-Latest	80.8	76.0	66.4	46.4	49.6	36.8	59.3	±49.1
5	GPT-4o	80.8	79.2	72.8	70.4	76.0	64.0	73.8	±43.9

When examining performance across specific error types with role-based and two-shot prompting, DeepSeek-Chat achieved an Avg. Acc of 83.8%, with strong results on MP at 93.4% and MB at 93.4%; but a moderate dip on MQ at 72.8%. DeepSeek-Reasoner topped the group at 86.6%, excelling on MB at 99.1% and MC at 98.2%, while notably struggling with MQo at 40.5%. DeepSeek-Coder averaged 81.4%, performing best on MP at 95.9% and MB at 94.0%, but showing weakness on MQ at 70.1%. Compared to two-shot prompting alone, role-based enhancements improved accuracies across most error types, such as DeepSeek-Chat's MCo rising from 80.0% to 86.1% and DeepSeek-Reasoner's KID from 89.8% to 92.0%; however, challenges persisted with MQo, e.g., DeepSeek-Reasoner dropped from 51.3% to 40.5%. These results indicate that the combined prompting strategy enhances error detection for most syntax errors, particularly for DeepSeek-Reasoner, which consistently outperformed others, while highlighting persistent difficulties with MQo across models. This suggests error-type-specific limitations, even with advanced prompting techniques. Grok-2-Latest achieved an Avg. Acc of 59.3%, with individual error type scores ranging from 88.7% on MC to 45.6% on MP. GPT-4o reached an Avg. Acc of 73.8%, performing best on MC at 90.3% and MB at 87.7%, while showing weakness on MQo at 7.0%. Table 6 presents each error type result with role-based and two-shot prompting.

Table 6. Acc for each error type using role-based and two-shot prompting.

No.	Model	MC	MP	MQo	MCo	MQ	MB	KID	Avg. Acc
1	DeepSeek-Chat	92.9	93.4	76.0	86.1	72.8	93.4	71.4	83.8
2	DeepSeek-Reasoner	98.2	94.2	40.5	93.2	87.6	99.1	92.0	86.6
3	DeepSeek-Coder	88.3	95.9	69.5	82.3	70.1	94.0	72.6	81.4
4	Grok-2-Latest	88.7	45.6	54.7	66.6	57.3	45.7	54.4	59.3
5	GPT-4o	90.3	83.6	7.0	77.1	86.0	87.7	78.9	73.8

5.3. Computational Time

We also measured the computational time for the 750 test runs under both prompting strategies. With two-shot prompting alone, DeepSeek-Chat completed in 01:41:20, DeepSeek-Reasoner in 05:49:58, DeepSeek-Coder in 01:35:16, and Grok-2-Latest in a notably faster 00:42:47, reflecting varying processing efficiencies, with Grok-2-Latest being the quickest despite its lower Acc. For the benchmark GPT-4o, computational time is not reported. Using role-based and two-shot prompting, DeepSeek-Chat improved to 01:32:40, suggesting a slight efficiency gain, while DeepSeek-Coder took 01:39:27, a minor increase from 01:35:16, and DeepSeek-Reasoner increased significantly to 10:19:10 from 05:49:58, indicating substantial computational overhead, possibly due to its higher Acc and the added complexity in handling role-based context. For Grok-2-Latest, the computational time decreased slightly from 00:42:47 with two-shot prompting to 00:41:51 with role-based and two-shot prompting, further underscoring its relative efficiency. Table 7 shows the time taken for each model.

Table 7. Computation time taken by each model.

No.	Model	Two-Shot Prompting	Role-Based and Two-Shot Prompting
1	DeepSeek-Chat	01:41:20	01:32:40
2	DeepSeek-Reasoner	05:49:58	10:19:10
3	DeepSeek-Coder	01:35:16	01:39:27
4	Grok-2-Latest	00:42:47	00:41:51
5	GPT-4o	Not mentioned	02:04:26

Figure 6 presents a series of confusion matrix heatmaps that illustrate the performance of each model at depths of varying context lengths. Panels (a), (c), (e), (g), and (i) reflect results under two-shot prompting only, while panels (b), (d), (f), (h), and (j) incorporate role-based and two-shot prompting, allowing for direct comparison of strategy effectiveness. The visualizations clearly show that all models experienced a gradual decline in Acc as context length increased, with deeper target depths compounding this effect, most notably in Grok-2-Latest (g), where Acc dropped sharply to as low as 16% at 16k tokens. DeepSeek-Reasoner with role-based and two-shot prompting (d) consistently maintained high Acc across all lengths and depths, especially in mid-range contexts (e.g., tokens 2k–4k), confirming its robustness. Furthermore, the benefit of role-based prompting is evident when comparing (a) to (b), (c) to (d), (e) to (f), and (g) to (h), where each model showed increased stability and Acc, especially at deeper target depths. For example, DeepSeek-Chat improved from 68% to 84% at 16k tokens and 0% depth, and DeepSeek-Coder showed better retention at deeper levels with role-based support. These heatmaps visually reinforce the findings discussed earlier, highlighting DeepSeek-Reasoner's superior adaptability and Grok-2-Latest's limitations, while showcasing how prompt engineering plays a critical role in preserving the Acc of the model in increasingly complex coding scenarios.

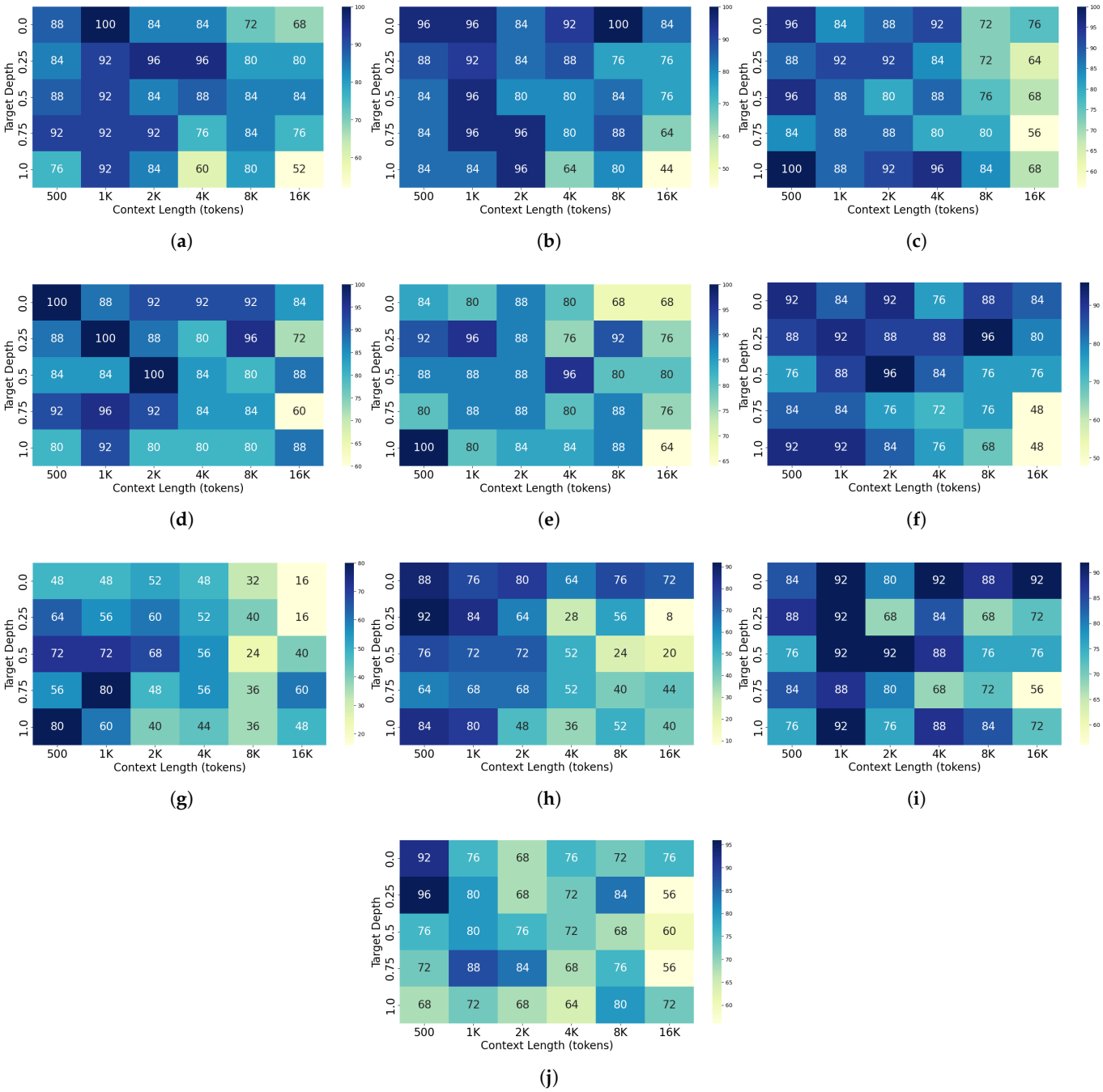

Figure 6. Confusion matrix of the models. (**a**) DeepSeek-Chat with two-shot prompting. (**b**) DeepSeek-Chat with role-based and two-shot prompting. (**c**) DeepSeek-Reasoner with two-shot prompting. (**d**) DeepSeek-Reasoner with role-based and two-shot prompting. (**e**) DeepSeek-Coder with two-shot prompting. (**f**) DeepSeek-Coder with role-based and two-shot prompting. (**g**) Grok-2-Latest with two-shot prompting. (**h**) Grok-2-Latest with role-based and two-shot prompting. (**i**) GPT-4o with two-shot prompting [14]. (**j**) GPT-4o with role-based and two-shot prompting.

6. Discussion

The experiment demonstrated that DeepSeek models consistently outperformed Grok-2-Latest at detecting syntax errors on code snippets with various lengths and depths. Through two-shot prompting, DeepSeek models achieved between 83.3% and 83.6% Avg. Acc, and through the addition of role-based and two-shot prompting, there was further improvement, with DeepSeek-Reasoner reaching the highest at 86.6%. Grok-2-Latest,

however, lagged far behind with a mere Avg. Acc of 50.2%, which suggests that it may not be best optimized for structured code understanding or syntax-level operations.

Though all tested models support increased context lengths of 64k to 128k tokens, the results report a dominating trend: the performance decreases as the token length increases. For example, DeepSeek-Chat showed a drop in Acc from 85.6% at 500 tokens to 72.0% at 16k tokens under two-shot prompting, demonstrating potential limits in attention span or memory. An exception was GPT-4o [14], which maintained a relatively stable performance across the different lengths, scoring an Avg. Acc of 81.2% with a low variance of ±5.5, thus making it a strong baseline.

The use of role-based prompting significantly improved detection rates for the various models, particularly DeepSeek-Reasoner, but at the expense of much longer runtimes. As indicated in Table 7, DeepSeek-Reasoner runtime doubled from 05:49:58 to 10:19:10, and there was evidently a trade-off between Acc and computational efficiency. This overhead can be attributed partly to the longer prompt length introduced by role-based instructions and partly to the model's internal reasoning process, which may expand during role-based prompting. These factors together highlight the trade-off between richer guidance and computational efficiency. Such a trade-off is an important factor in real deployment settings, where runtime limitations tend to restrict practical deployments.

Despite the gain, even top-performing models were afflicted with some types of errors such as MQo, which shows that some patterns of syntax are still difficult for LLMs to capture regardless of the prompting approach. It is likely that subtle syntax errors often do not strongly disrupt the surrounding context, making them less noticeable to the model. These persistent vulnerabilities also illustrate the value of additional targeted training or fine-tuning on rare or complex syntax errors.

While Avg. Acc indicates peak model performance, high STD values reveal variability and potential instability. For example, DeepSeek models achieve high Avg. Acc but shows large STD, suggesting that their performance is less predictable across scenarios. In contrast, GPT-4o demonstrates lower STD, reflecting more stable and reliable behavior. Considering both Acc and STD allows a more comprehensive evaluation, highlighting the trade-off between peak performance and consistency in model behavior.

Overall, the findings demonstrate that DeepSeek models possess a great capacity for syntax error detection, particularly for short or comparatively complex inputs, and GPT-4o has a balanced performance and stability. However, practical considerations such as runtime, input length sensitivity, and vulnerability to specific errors must be addressed to enable successful application in real-time or large-scale coding environments.

To address potential data contamination and the limited diversity of error types in the original test set, we constructed an additional self-organized test set that includes novel error types. This dataset was deliberately kept simple in structure, yet distinct from the Alpaca dataset evaluation. In this way, we ensured that the errors tested were not trivially memorized during pre-training. Specifically, we focused on three new types of syntax errors: indentation error, which arises when code blocks are not properly indented according to Python's syntax rules; invalid assignment, which occurs when assignment is attempted to an invalid target such as a number; and missing as, a frequent error in Python exception handling or context management statements when the as keyword is omitted. With this dataset, we evaluated the two top-performing models under each prompting strategy: DeepSeek-Coder and DeepSeek-Reasoner. For this additional evaluation, the setup was intentionally lightweight for each combination of code length and error depth; only three iterations were generated and tested, ensuring a manageable yet illustrative assessment. The resulting analysis provides further evidence of the models' robustness

when facing error types beyond the commonly tested ones, thus offering a more rigorous and diversified assessment of their syntax error detection capabilities.

The results on the self-organized test set indicate that DeepSeek-Reasoner consistently outperformed DeepSeek-Coder across most code lengths and error types. In terms of overall Acc across token lengths, DeepSeek-Reasoner achieved an Avg. Acc of 81.1%, with a peak of 100% at 8k tokens and a low of 60% at 16k tokens, whereas DeepSeek-Coder averaged 76.6%, ranging from 53.3% at 4k tokens to 86.6% at 16k tokens. Examining performance by error type, DeepSeek-Reasoner performed best on missing as at 90.0%, followed by indentation error at 83.2%, and struggled most with invalid assignment at 70.1%. DeepSeek-Coder achieved perfect Acc on missing as at 100%, moderate performance on invalid assignment at 75.5%, and was weakest on indentation error at 54.3%. These results highlight that while both models can handle novel syntax errors, DeepSeek-Reasoner demonstrates more consistent robustness, particularly across error types and varying code lengths. Tables 8 and 9 present Acc of models in each length and error type.

Table 8. Acc scores of models on the self-organized test set.

No.	Model	500	1k	2k	4k	8k	16k	Avg. Acc	STD
1	DeepSeek-Reasoner	93.3	80.0	73.3	80.0	100	60.0	81.1	±39.3
2	DeepSeek-Coder	73.3	80.0	73.3	53.3	93.3	86.6	76.6	±42.5

Table 9. Acc for each error type on the self-organized test set.

No.	Model	Indentation Error	Invalid Assignment	Missing as	Avg. Acc
1	DeepSeek-Reasoner	83.2	70.1	90.0	81.1
2	DeepSeek-Coder	54.3	75.5	100	76.6

7. Conclusions

The goal of this research is to assess how effectively LLMs can identify and categorize syntax errors in large Python code, with the aim of measuring their performance under varying input lengths, depths, error types, and prompting techniques. The results of this research highlight both the strengths and limitations of existing LLMs in identifying syntax errors in large-scale Python codebases. DeepSeek-Reasoner reported the best Avg. Acc among the models tested, especially when utilizing role-based prompting combined with two-shot prompting, indicating the usefulness of contextual augmentation for model guidance. While the DeepSeek models performed well in syntax error detection, several limitations were observed. Firstly, all models were vulnerable to longer input sequences, with reduced Acc when context length was raised despite having large window capacities. This suggests that effective utilization of longer contexts remains an open problem. Also, the increased efficiency achieved with role-based prompting was at the expense of a notable runtime increase, especially for DeepSeek-Reasoner, which can restrict its real-world usage to settings where time and resources are not limited. Another notable limitation is the inconsistent performance across certain error types, specifically MQo, which persisted across models and prompting strategies. This indicates that current LLMs may still lack robust internal representations for handling rare or structurally complex syntax patterns. Therefore, while models are effective in controlled settings, their deployment in real-world coding environments requires further optimization to address issues related to efficiency, generalization, and error-specific handling. These findings open up avenues for additional research in AI-driven software maintenance and building more robust models that can deal with large-scale, complicated programming environments. Future research can be conducted by expanding the analysis to multi-language codebases other than Python, or by

increasing the input size to evaluate the performance at the scale of complete production-level repositories. Future research may also involve testing additional LLMs to explore how newer or alternative models perform in syntax error detection tasks. These directions would provide a fuller picture of model robustness and generalizability across a variety of software development scenarios.

Author Contributions: Conceptualization, N.A. and A.A.; investigation, N.A. and A.A.; methodology, N.A. and A.A.; project administration, N.A.; resources, N.A. and A.A.; supervision, A.A.; validation, N.A. and A.A.; writing—original draft, N.A. and A.A.; writing—review and editing, N.A. and A.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Data Availability Statement: The data used in this study are publicly available and can be accessed at the following Hugging Face repository: https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca (accessed on 20 July 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLMs	Large Language Models
MC	Missing Colon
MQ	Missing Quotation
MP	Missing Parenthesis
MCo	Missing Comma
MQ	Mismatched Quotation
MB	Mismatched Brackets
KID	Keyword as Identifier

References

1. Gore, D.V.; Binoj, M.; Borate, S.; Devnani, R.; Gopale, S. Syntax Error Detection and Correction in Python Code using ML. *Grenze Int. J. Eng. Technol. (GIJET)* **2023**, *9*, 296.
2. Alasmari, O.A.; Singer, J.; Bikanga Ada, M. Do current online coding tutorial systems address novice programmer difficulties? In Proceedings of the 15th International Conference on Education Technology and Computers, Barcelona, Spain, 26–28 September 2023; pp. 242–248.
3. Aniche, M.; Maziero, E.; Durelli, R.; Durelli, V.H. The effectiveness of supervised machine learning algorithms in predicting software refactoring. *IEEE Trans. Softw. Eng.* **2020**, *48*, 1432–1450. [CrossRef]
4. Zhu, Q.; Sun, Z.; Xiao, Y.a.; Zhang, W.; Yuan, K.; Xiong, Y.; Zhang, L. A syntax-guided edit decoder for neural program repair. In Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, 23–28 August 2021; pp. 341–353.
5. Wu, L.; Li, F.; Wu, Y.; Zheng, T. Ggf: A graph-based method for programming language syntax error correction. In Proceedings of the 28th International Conference on Program Comprehension, Seoul, Republic of Korea, 13–15 July 2020; pp. 139–148.
6. Li, T.; Zhang, Y. Multilingual code refactoring detection based on deep learning. *Expert Syst. Appl.* **2024**, *258*, 125164. [CrossRef]
7. Pandhare, H.V. From Test Case Design to Test Data Generation: How AI is Redefining QA Processes. *Int. J. Eng. Comput. Sci.* **2024**, *13*, 26737–26757. [CrossRef]
8. Gao, S.; Gao, C.; He, Y.; Zeng, J.; Nie, L.; Xia, X.; Lyu, M. Code structure–guided transformer for source code summarization. *ACM Trans. Softw. Eng. Methodol.* **2023**, *32*, 1–32. [CrossRef]
9. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [CrossRef]
10. Liu, J.; Chen, Y.; Liu, M.; Peng, X.; Lou, Y. Stall+: Boosting llm-based repository-level code completion with static analysis. *arXiv* **2024**, arXiv:2406.10018.

11. Campos, V. Bug Detection and Localization using Pre-trained Code Language Models. In Proceedings of the INFORMATIK 2024, Wiesbaden, Germany, 24–26 September 2024; Gesellschaft für Informatik eV: Bonn, Germany, 2024; pp. 1419–1429.
12. Li, Y.; Parsert, J.; Polgreen, E. Guiding enumerative program synthesis with large language models. In Proceedings of the International Conference on Computer Aided Verification, Montreal, QC, Canada, 24–27 July 2024; Springer: Cham, Switzerland, 2024; pp. 280–301.
13. Kamradt, G. Needle In A Haystack—Pressure Testing LLMs. 2023. Available online: https://github.com/gkamradt/LLMTest_NeedleInAHaystack (accessed on 20 July 2025).
14. Lee, H.; Sharma, S.; Hu, B. Bug in the code stack: Can llms find bugs in large python code stacks. *arXiv* **2024**, arXiv:2406.15325. [CrossRef]
15. Anthropic, A. Introducing the Next Generation of Claude. 2024. Available online: https://www.anthropic.com/news/claude-3-family (accessed on 1 July 2025).
16. Team, G.; Georgiev, P.; Lei, V.I.; Burnell, R.; Bai, L.; Gulati, A.; Tanzer, G.; Vincent, D.; Pan, Z.; Wang, S.; et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv* **2024**, arXiv:2403.05530. [CrossRef]
17. Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; Ruan, C.; et al. Deepseek-v3 technical report. *arXiv* **2024**, arXiv:2412.19437.
18. xAI. *Grok-2 Beta Release*; xAI: San Francisco Bay, CA, USA, 2024.
19. Rahman, M.M.; Watanobe, Y.; Nakamura, K. Source code assessment and classification based on estimated error probability using attentive LSTM language model and its application in programming education. *Appl. Sci.* **2020**, *10*, 2973. [CrossRef]
20. Kanoutas, T.; Karanikiotis, T.; Symeonidis, A.L. Enhancing code readability through automated consistent formatting. *Electronics* **2024**, *13*, 2073. [CrossRef]
21. Wang, J.; Li, L.; Liu, K.; Du, X. Detecting and Explaining Python Name Errors. *Inf. Softw. Technol.* **2025**, *178*, 107592. [CrossRef]
22. Yasunaga, M.; Liang, P. Break-it-fix-it: Unsupervised learning for program repair. In Proceedings of the International Conference on Machine Learning, Virtual, 18–24 July 2021; PMLR: Cambridge, MA, USA, 2021; pp. 11941–11952.
23. Ahmed, T.; Ledesma, N.R.; Devanbu, P. Synshine: Improved fixing of syntax errors. *IEEE Trans. Softw. Eng.* **2022**, *49*, 2169–2181. [CrossRef]
24. Lutellier, T.; Pham, H.V.; Pang, L.; Li, Y.; Wei, M.; Tan, L. Coconut: Combining context-aware neural translation models using ensemble for program repair. In Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, 18–22 July 2020; pp. 101–114.
25. Dinella, E.; Dai, H.; Li, Z.; Naik, M.; Song, L.; Wang, K. Hoppity: Learning graph transformations to detect and fix bugs in programs. In Proceedings of the International Conference on Learning Representations (ICLR), Virtual, 26 April–1 May 2020.
26. Berabi, B.; He, J.; Raychev, V.; Vechev, M. Tfix: Learning to fix coding errors with a text-to-text transformer. In Proceedings of the International Conference on Machine Learning, Virtual, 18–24 July 2021; PMLR: Cambridge, MA, USA, 2021; pp. 780–791.
27. Phung, T.; Cambronero, J.; Gulwani, S.; Kohn, T.; Majumdar, R.; Singla, A.; Soares, G. Generating high-precision feedback for programming syntax errors using large language models. *arXiv* **2023**, arXiv:2302.04662. [CrossRef]
28. Yang, Z.; Wang, S.; Yan, Y.; Deng, Y. Why Stop at One Error? Benchmarking LLMs as Data Science Code Debuggers for Multi-Hop and Multi-Bug Errors. *arXiv* **2025**, arXiv:2503.22388.
29. Prenner, J.A.; Robbes, R. Automatic program repair with openai's codex: Evaluating quixbugs. *arXiv* **2021**, arXiv:2111.03922.
30. Bisht, T. python-code-instructions-18k-alpaca. 2023. Available online: https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca (accessed on 20 July 2025).
31. Rajaraman, N.; Jiao, J.; Ramchandran, K. Toward a theory of tokenization in llms. *arXiv* **2024**, arXiv:2404.08335. [CrossRef]
32. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901.
33. Bansal, P. Prompt engineering importance and applicability with generative AI. *J. Comput. Commun.* **2024**, *12*, 14–23. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.